#6/#7 If the new entry is closer (by whatever definition of "closer" is assumed) then update the temporary variables with the results.
#8 Return a pair with the closest entry and the node containing it, for the caller's benefit.

So, to complete our description of the `delete` method, we can start from `findClosestEntryInNodesList`: listing 10.16 shows the pseudocode for the method, that is just another linear search within a list of nodes, with the goal of finding the closest entry contained in any of the nodes in the list; notice that we also return the parent node because it will be needed by the caller.

Next, listing 10.17 describes method `borrowFromSibling`, which moves an entry (respectively a point, for leaves, or a child node, for internal nodes) to the node that currently violates the minimum points/children invariant, taking it from one of its siblings. Obviously, we need to choose a sibling that has more than `m` entries, to avoid moving the issue around (the sibling will have one less entry afterwards, and we don't want it to violate the invariant itself!): for this implementation, we will assume that all elements of the list `siblings`, passed in input, are non-null nodes with at least `m+1` entries, and of course also that `siblings` is not empty. If you are implementing this code in a language that support assertions, you might want to add an assert to verify these conditions[14].

Listing 10.17 The SsNode::borrowFromSibling method